ISTQB® Certified Tester

Advanced Level — **Security Tester** (CT-SEC, Syllabus 2016)

CHAPTER 8

Security Testing Tools

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	8.1 Types and Purposes of Security Testing Tools · 8.2 Tool Selection
Learning objectives	4 examinable (AS-8.1.1 – AS-8.2.3) · K2 = 3 · K4 = 1
Recommended teaching time	55 minutes — **5 points**, **5.5 points per teaching hour — the best return in the syllabus**
Questions in the exam	**3 of 45 questions · 5 of 80 points.** K2 = 2 · **K4 = 1**. Guaranteed: **AS-8.1.1** (K2) and **AS-8.2.1** (K4). One K2 from 8.2.2/8.2.3
Exam format	45 questions · **80 points · 120 minutes** (150 with the 25% non-native-language allowance) · pass mark 65% = **52 points** · K2 = 1 pt · K3 = 2 pts · K4 = 3 pts
Keywords	none listed by the syllabus
Official sample-exam questions	**#41 – #43** (1 + 3 + 1 points)

Built exclusively from the ISTQB® Advanced Level Security Tester syllabus (2016), the official Sample Exam (questions and answers, GA March 2016) and the ISTQB® exam structure tables.

Study this single file to master Chapter 8.

Contents

Section 1 — Syllabus Explanation....4
AS-8.1.1 (K2) — Types and Purposes of Security Testing Tools....4
AS-8.2.1 (K4) — Analyzing and Documenting Security Testing Needs....5
AS-8.2.2 (K2) — Issues with Open Source Tools....6
AS-8.2.3 (K2) — Evaluating a Tool Vendor's Capabilities....7
Section 2 — Key Concepts....8
2.1 The storyline of Chapter 8....8
2.2 Tools: types, needs and selection....8
2.3 Fuzzy points clarified....10
2.4 Chapter 8 keyword glossary....10
Section 3 — Tables and Summaries....11
Section 4 — Demo Questions with Answers....12
Part A — The 3 official Chapter 8 questions....12
Part B — 12 additional syllabus-derived questions....13
Most Important Questions From This Chapter....15
Section 5 — Exam Tricks....17
5.1 Frequently tested topics (evidence-based)....17
5.2 Common distractors observed....17
5.3 Keyword recognition (map words → answers)....17
5.4 Typical traps....17
5.5 How to eliminate wrong answers....17
5.6 Scenario-solving shortcuts....18
5.7 Common candidate mistakes....18
5.8 Mnemonics that carry the whole chapter....18
5.9 Chapter 8 mind map....18
Section 6 — Revision Notes....19
6.1 One-page chapter summary....19
6.2 Key definitions to memorize....19
6.3 High-priority concepts (study first)....19
6.4 Last-minute revision sheet....19
6.5 Memorization checklist....19
6.6 Exam-day quick review....19

In Word, right-click the table of contents and choose "Update Field" → "Update entire table" to refresh the page numbers.

How to use this guide

Section 1 explains the four learning objectives. **Section 2** explains every key concept from the ground up — plain words, how it works, a worked example, the exam angle, and what it is easily confused with — and ends with the chapter's **fuzzy points clarified**. If the syllabus wording in Section 1 feels dense, read the matching Section 2 card first. **Section 3** compares open source and commercial tools and maps needs to capabilities. **Section 4** is the question bank, **Section 5** exam tactics, **Section 6** is the revision sheet for the night before.

What makes Chapter 8 unusual: it is the **best value per hour on the paper**, and its **3-point K4 is a firewall testing scenario** — the answer comes from **Chapter 5** (malformed packets and network fuzzing).

The rules that decide it: dynamic security tools probe the system, not just the application; for an **open source (GPL) tool there is no vendor**, so maintenance depends on **your team's capability**; for a commercial tool, judge the vendor's **update frequency** against evolving threats.

Suggested schedule: one session for Sections 1–3, one for Sections 4–5; redo missed questions and re-read Section 2's fuzzy points; Section 6 the night before.

Section 1 — Syllabus Explanation

Chapter 8, **Security Testing Tools**, carries **4 learning objectives** across two sub-chapters (8.1 Types and Purposes · 8.2 Tool Selection). The Exam Structure Table allocates **3 questions** and **5 of the 80 points**. Teaching time is **55 minutes** — **5.5 points per teaching hour, the best return in the syllabus**.

The chapter's **3-point K4 (AS-8.2.1)** asks you to analyze security testing needs that tools should address. In the official Sample Exam that question is a **firewall test scenario** — so it draws on **Chapter 5** knowledge as much as Chapter 8. The other two questions are K2: one guaranteed on **static vs dynamic tools (AS-8.1.1)**, one from **open source issues** or **vendor capabilities**.

LO ID	K	Objective (syllabus statement)	Qs	Pts
AS-8.1.1	K2	Explain the role of static and dynamic analysis tools in security testing.	**1**	1
AS-8.2.1	**K4**	Analyze and document security testing needs to be addressed by one or more tools.	**1**	**3**
AS-8.2.2	K2	Understand the issues with open source tools.	1 of 2†	1
AS-8.2.3	K2	Understand the need to evaluate the vendor's capabilities to update tools on a frequent basis to stay current with security threats.	1 of 2†	1
Chapter 8 total	—	**K2 = 2 · K4 = 1**	**3**	**5**

*† One K2 drawn from AS-8.2.2 / 8.2.3 — the Sample Exam drew **8.2.2** (Q43, a GNU GPL tool).*

Exam allocation you must know

Chapter 8 = **3 questions / 5 points**; Sample Exam **#41 to #43** (1 + 3 + 1 points).

The one thing that decides this chapter

Open source has no vendor — so the question is your own team's capacity. For a GNU GPL tool (Q43), the maintenance consideration is **the technical capabilities of your team to support and customize the tool**; every option mentioning a **vendor**, **updates from the vendor** or **licence and support contract cost** is wrong. For commercial tools the mirror rule applies: **evaluate the vendor's ability to update frequently** to keep pace with threats.

AS-8.1.1 (K2) — Types and Purposes of Security Testing Tools

Original syllabus statement

Explain the role of static and dynamic analysis tools in security testing.

Where security tools came from

- **Exploits devised by the hacking community drove the development of security testing tools.** Early tools (e.g. **password cracking**) were **shared and improved** by hackers; **dedicated** to tasks and environments; **usability was not an issue** for technical users.
- Some **hacker tools became the basis for legitimate security testing tools**. Example: **John the Ripper** — an early **open source password cracker** for Unix, now **used legitimately to detect weak Unix passwords**.
- As major suppliers built security tools, they gained **broader functionality and better usability** — but **more complex configuration and implementation concerns**.
- Frameworks such as **Nessus and Metasploit** emerged as **open source** with expanded functionality and in some cases **an easily learnable GUI**.

Today's landscape

- **The number of tools is massive** — a dedicated tool exists for almost any environment or task, open source or licensed.
- The challenge: most are **intelligent systems deploying non-standardized tests** — they may use **different test data, implementations and interpretations of results**. Since the same defense or vulnerability can be **implemented in different ways**, **selecting and using tools is a challenge**.
- Tools **automate the evaluation of defenses** and **detect known types of vulnerabilities**.
- Lists of categorized tools: **Web Application Security Consortium**, **OWASP**; frameworks **BackTrack / Kali Linux** offer other classifications; **sectools.org**. **Commercial tools are few compared with open source.** The advanced tester **maintains and updates his or her own tool list**.

Static vs dynamic in security testing

	Static analysis tools	Dynamic analysis tools
Advantage	**Can be performed very early in the lifecycle**; available for **most languages**; usually **report on security aspects**	Exercise the running system
Definition (security context)	Test activities while the object **is not in operational mode**	Running probes — but **often probing the system rather than the application under test**
The twist	—	A **dynamic** security tool may perform a **static scan** — e.g. **of a database**. If **the entire system** is the test object, they truly are dynamic tools

How the official exam used this objective

Sample Q41: how do dynamic **security** analysis tools differ from general dynamic analysis tools? **The security tools probe the system rather than just the application under test.** Rejected: "work the same in dynamic or static mode" (there are **both** dynamic and static security tools); "better at memory leaks" (**general** dynamic tools detect leaks); "tailored to the language" (true of **all static analysis tools**).

AS-8.2.1 (K4) — Analyzing and Documenting Security Testing Needs

Original syllabus statement

Analyze and document security testing needs to be addressed by one or more tools.

Documents that can form a test basis for security testing

#	Test basis
1	**The organization's security policy**
2	**The organization's test policy**
3	**Results of threat and risk analysis** for the actual system/project
4	**Requirements and other system specifications**
5	**System architecture and design**
6	**Security (test) strategy**
7	**The system or application under test**
8	**Known security threats, exploits and vulnerabilities**
9	**User profiles**

Requirements and design documents should show **how data is protected**, giving an overview of:

- **Interfaces to be tested (including the GUI)**
- **Protocols and standards to be verified**
- **Web coding guidelines** promoting secure coding to be used
- **System component configurations to be verified (hardened)**

Also determine **whether security testing will be a development activity or a maintenance/operations activity**. **All this leads to the requirements for the security test tool set.**

How the official exam used this objective (K4, 3 points)

Sample Q42: you have **reviewed the firewall implementation plan, verified the vendor configuration and conducted port scanning**; the organization is worried about **DoS** (it had one with the old firewall). What testing detects unexpected behavior exploitable by DoS? **Create tests that send malformed network packets or fuzzed data and see if they are detected and rejected by the firewall.**

Rejected: **stress test the servers for failover** and **test whether encryption is fast enough under a DoS** (the goal is to **prevent the attack**, not let it through); **software component hardening** (helps components, **not the firewall**). This is **§5.4.2** knowledge applied to a tool-needs question: the stated need (unexpected behaviour under DoS) determines the tool capability (packet crafting, network fuzzing).

K4 method — from need to tool capability

Read the scenario for **what is already done** (strike it), **the remaining risk** (the stated worry), and **the test basis items** it touches (architecture, configurations, protocols, interfaces). The answer names the **tool capability** that addresses that remaining risk — and nothing that addresses a different goal (performance, failover, a different layer).

AS-8.2.2 (K2) — Issues with Open Source Tools

Original syllabus statement

Understand the issues with open source tools.

- Many security tools are **open source**, under licences that **allow free use and modification of the source code**. (The Advanced Test Manager syllabus covers open source issues in full.)
- **Not all organizations can use them** — **regulatory compliance** may force **commercial or otherwise certified tools only**.
- Often **free to obtain**, but the organization **may need technical capacity for support and specific configuration**; lacking it, **cost may be incurred to obtain it from the software's developer**.
- **Manuals**, if any, target **a technical audience** and **seldom cover all functionality**; **YouTube** is an additional source.

Aspects of an ROI calculation for an open source tool

Aspect
Limited scope of the tool — usually no further functionality is offered
Time to learn to administer, configure and use it
Time invested in user forums and groups during the lifecycle
Time to update and upgrade (and the internal policy on upgrades)
Future direction — some tools **go away or go commercial**
Responsiveness of the support community

Licences: most businesses need **one or a few** security tool licences; only large companies consider more. The number depends on **functional areas delivered** (web application, web services, code analysis…), **frequency and time of use**, and **the number of security testers**.

How the official exam used this objective

Sample Q43: for a tool under the **GNU General Public License**, important maintenance consideration? **Technical capabilities of your team to support and customize the tool for your environment.** Rationale: **the GNU license is free and it is an open source community so there is no vendor**; vendor reliability and vendor updates are wrong for that reason; licence and support-contract cost is wrong because **the tool is free** (though customization has development costs).

AS-8.2.3 (K2) — Evaluating a Tool Vendor's Capabilities

Original syllabus statement

Understand the need to evaluate the vendor's capabilities to update tools on a frequent basis to stay current with security threats.

A purchased tool's vendor should offer services that **allow the security testing service to start and grow to the needed level of internal support**. Attributes to evaluate:

Attribute	Examples
Types of licences	**Fixed / desk / floating / token**
Licence scalability	**Per functional area, number of licences**
Helpdesk / support facilities	**Hours of support**
Forum / user community	—
Update frequency	**Staying current with security threats** — the LO's focus
Administration and user manuals	—
Support and maintenance contracts	—

Section 2 — Key Concepts

Section 1 gives the syllabus wording objective by objective. This section explains what that wording means. Each card builds one idea in order: plain words first, then how it works, an example, how the exam uses it, and what it is easily confused with.

2.1 The storyline of Chapter 8

Chapter 8 asks three questions about tools. **What kinds are there?** (Many, mostly open source, grown out of hacker tools, running non-standard tests, both static and dynamic.) **How do you decide what you need?** (Start from the test basis, the system's interfaces, protocols and configurations, and the risk still uncovered, then pick the capability that addresses that risk.) **Who keeps the tool working?** (For **open source**, your **own team**, because there is no vendor. For **commercial** tools, judge the **vendor's ability to update quickly** as threats change.)

2.2 Tools: types, needs and selection

Static and dynamic security tools (AS-8.1.1, guaranteed K2)	
In plain words	**Static** tools examine something **that is not running** (source code, configuration files, a database). **Dynamic** tools exercise a **running** system. Security tools add one twist: dynamic **security** tools usually **probe the whole system** (network, server, platform), not just the application under test.
How it works	**History: hacker exploits drove the creation of security tools. John the Ripper** began as an open source password cracker and is now used **legitimately to find weak Unix passwords**. **Nessus** and **Metasploit** grew into open source frameworks, some with an easy GUI. Commercial tools added functionality and usability, but also configuration complexity. **Today:** a **massive** number of tools, mostly open source, from lists such as **WASC, OWASP, Kali Linux (formerly BackTrack) and sectools.org**. Because tools run **non-standardized tests** (different data, implementations and interpretations), **choosing and using them is a challenge**. Advanced testers **keep their own up-to-date tool list**. **Tools automate the evaluation of defenses and detect known types of vulnerabilities.** **Static tools:** can run **very early in the lifecycle**, exist for **most languages**, and usually **report on security aspects**. **Dynamic security tools** probe **the system rather than only the application**. Some dynamic tools even perform a **static scan** (for example, of a database). When the **whole system** is the test object, they are truly dynamic.
Example	A static analyzer flags a hard-coded database password in the source on day 3 of coding. Months later, a dynamic vulnerability scanner run against the test server finds an outdated SSL library in the **web server**, a finding about the platform, not the application code.
Exam angle	Official Q41: how do dynamic **security** analysis tools differ from general dynamic analysis tools? → **they probe the system rather than just the application under test.** Memory-leak detection is a feature of **general** dynamic tools. "Tailored to the language" describes **static** tools in general.
Don't confuse	"Dynamic" describes **how** the tool works (running object), not **what** it scans. A dynamic security tool can still scan something **statically**.

Analyzing and documenting the needs a tool must meet (AS-8.2.1, guaranteed K4)	
In plain words	Do not start from a tool catalogue. Start from **what must be tested and what risk is still uncovered**, write it down as **requirements for the tool set**, and then choose tools whose **capabilities** match.

Analyzing and documenting the needs a tool must meet (AS-8.2.1, guaranteed K4)	
How it works	**Nine possible test-basis documents:** the **security policy** · the **test policy** · **threat and risk analysis results** · **requirements and specifications** · **architecture and design** · the **security (test) strategy** · **the system under test** itself · **known threats, exploits and vulnerabilities** · **user profiles**. **From requirements and design, extract:** the **interfaces** to test (including the GUI) · the **protocols and standards** to verify · the **web coding guidelines** to check · the **component configurations** to verify (hardening). **Decide** whether security testing is a **development** activity or a **maintenance/operations** activity, because that changes who uses the tools and how. **All of this becomes the requirements for the security test tool set.**
Method	(1) strike everything the scenario says is **already done** · (2) find the **remaining risk** (the stated worry) · (3) pick the tool **capability** that tests that risk, and nothing aimed at a different goal (performance, failover, a different layer).
Example	Official Q42: the firewall plan has been reviewed, the vendor configuration verified, and ports scanned. The worry is **DoS**, which happened with the old firewall. Remaining need: unexpected behaviour that DoS could exploit → **send malformed packets or fuzzed data and check the firewall detects and rejects them** (packet-crafting or network-fuzzing capability).
Exam angle	In Q42, reject failover stress tests and "is encryption fast enough during DoS" (they **let the attack through** instead of preventing it), and component hardening (it helps **components**, not the **firewall**). This question uses **Chapter 5 §5.4.2** knowledge.
Don't confuse	A **need** is a test goal ("detect unexpected firewall behaviour under malformed input"). A **tool** is a means. K4 options often name a real, useful tool that serves a **different** need.

Open source tools: free to get, not free to own (AS-8.2.2, K2 pool)	
In plain words	Many security tools are **open source**: you may **use and modify** them for free. But **there is no vendor**, so **your own team** must have the **technical capacity to support, configure and customize** them, or pay someone (possibly the tool's developer) for it.
How it works	**Regulation may forbid them:** some organizations must use **commercial or certified** tools only. **Manuals**, if they exist, are written for a **technical audience** and **seldom cover everything**. **YouTube** is an extra source. **Return-on-investment aspects:** the **limited scope** of the tool · **time to learn** to administer and use it · **time spent in forums and user groups** · **time to update and upgrade** (and your internal upgrade policy) · the tool's **future direction** (some **disappear or go commercial**) · how **responsive** the **support community** is. **How many licences:** most businesses need **one or a few**. The number depends on the **functional areas** covered (web apps, web services, code analysis…), **how often and how long** the tools are used, and the **number of security testers**.
Example	A team adopts a GPL-licensed scanner. It costs nothing to download, but the team spends three weeks learning to configure it for their single-sign-on setup, writes custom plug-ins, and must track community releases themselves. Those weeks are the real cost.
Exam angle	Official Q43: for a **GNU GPL** tool, the important maintenance consideration → **the technical capabilities of your team to support and customize the tool for your environment.** Any option about a **vendor's** reliability or updates, or about **licence and support-contract costs**, is wrong: the licence is free and there is no vendor.
Don't confuse	**Free licence ≠ zero cost.** Customization and support still cost time and money.

Evaluating a commercial vendor: can it keep up with threats? (AS-8.2.3, K2 pool)	
In plain words	When you **buy** a tool, the vendor's services must let your security testing **start and grow** to the level of internal support you need. Above all, the vendor must **update the tool often enough to stay current with new threats**. A security tool that does not know this month's attacks is out of date.
How it works	**Attributes to evaluate: licence types** (fixed, desk, floating, token) · **licence scalability** (per functional area, number of licences) · **helpdesk and support** (hours of support) · **forum and user community** · **update frequency** · **administration and user manuals** · **support and maintenance contracts**.
Example	Two commercial scanners are compared. Scanner A releases new vulnerability checks within 48 hours of major disclosures. Scanner B updates quarterly. For a public-facing web estate, A wins on the attribute this objective is about, even if B's floating licences are cheaper.
Exam angle	If the stem is about **staying current with security threats**, the answer is the vendor's **update frequency**. If the stem is about **open source**, there is no vendor to evaluate, so look at your **own team's capacity**.
Don't confuse	**Floating** licence = a shared pool used concurrently. **Token** licence = usage-based units. **Desk** licence = tied to a workstation. **Fixed** licence = tied to a named user or machine. You only need to recognise the four names.

2.3 Fuzzy points clarified

Places where the wording is easy to misread

"Dynamic tools ... often probing the system rather than the application" is the syllabus's distinction between **security** dynamic tools and **general** dynamic analysis tools. It does not mean dynamic security tools never test the application.

The need → capability table (Section 3, table 3.2) is this guide's mapping. The syllabus names John the Ripper, Nessus and Metasploit, but does not map tools to needs.

The licence-type explanations in section 2.2 are general industry meanings. The syllabus only lists the names: fixed, desk, floating and token.

Chapter 8's K4 is really a Chapter 5 question. Revise the firewall tests (port scan, malformed packets and fuzzing, fragmentation, WAF encoding) before attempting it.

2.4 Chapter 8 keyword glossary

The syllabus lists **no keywords** for Chapter 8. Useful working terms: **static analysis tool** (analyzes artifacts not in operational mode; early), **dynamic analysis tool** (probes a running system), **vulnerability scanner** (Chapter 5), **ROI** (return on investment in a tool), **floating licence** (shared pool of concurrent licences), **token licence** (usage-based licence units).

Section 3 — Tables and Summaries

3.1 Open source vs commercial — what to evaluate

Concern	Open source	Commercial
Support	**Your team's capacity**; community responsiveness	Vendor helpdesk hours, contracts
Updates	Time to update/upgrade; internal upgrade policy	**Vendor update frequency** vs threats
Cost	Free to obtain; learning, forum, customization time	Licence types and scalability
Documentation	Technical, incomplete; YouTube	Administration and user manuals
Longevity	May **go away or go commercial**	Vendor viability (implicit)
Compliance	May be **forbidden** by regulation	Certified tools may be required

3.2 Need → tool capability (illustrative)

Need in the scenario	Capability
Code-level vulnerabilities early	Static analysis / security-oriented analyzer
Firewall unexpected behaviour (DoS)	Malformed packet generation, network fuzzing
Firewall policy implemented?	Port scanner
Weak passwords on Unix	Password auditing (e.g. John the Ripper)
Known vulnerabilities across many hosts	Vulnerability scanner
Exploitability of a known vulnerability	Exploitation framework (e.g. Metasploit)

Table 3.2 links Chapter 8's selection logic to tests named in Chapters 3 and 5; the syllabus names John the Ripper, Nessus and Metasploit but does not map tools to needs.

Reproduce from memory before moving on

Dynamic security tools probe the system — the guaranteed K2.

Nine test-basis items and the **need → capability** method — the 3-point K4.

Open source = no vendor; your team and **vendor update frequency** — the pooled K2.

Section 4 — Demo Questions with Answers

*Questions 1–3 are the **official ISTQB CTAL-SEC Sample Exam questions for Chapter 8** — #41 to #43 — with official answers and rationale. LO tags are inferred and match the Exam Structure Table. Questions 4–15 are derived strictly from the syllabus. Correct options are marked ✔.*

Part A — The 3 official Chapter 8 questions

Q1. OFFICIAL · Sample Exam #41 (1 pt)

In what way are dynamic security analysis tools different from general dynamic analysis tools?

a) The security tools probe the system rather than just the application under test ✔

b) The security tools work the same in dynamic or static mode

c) The security tools are better suited to detect problems such as memory leaks

d) The security tools need to be tailored to the language in which the application is implemented

Correct answer: a | LO: AS-8.1.1 (K2) | Syllabus §8.1

Why: (a) — official answer. (b) **there are both dynamic and static analysis security tools**. (c) **memory leaks are detected by the general dynamic analysis tools, not the security specific ones**. (d) **this is true of all static analysis tools**. Syllabus §8.1: **it is not unusual for the security dynamic testing tools to probe the system rather than the application under test**.

Q2. OFFICIAL · Sample Exam #42 (3 pts)

You have been given the job of testing the organization's firewall. You have reviewed the implementation plan and steps, verified that the configuration has been set up as instructed by the firewall vendor and have conducted port scanning. Your organization is particularly concerned about denial of service (DOS) attacks, particularly since they had one when the old firewall was in place. What type of testing should you conduct to help detect unexpected behavior that could be exploited by a DOS attack?

a) Create tests that will send malformed network packets or fuzzed data and see if they are detected and rejected by the firewall ✔

b) Implement automated tests to stress test the servers to test the failover capabilities

c) Test the encryption and decryption algorithms to determine if they are fast enough to deal with the load of a DOS attack

d) Conduct software component hardening to ensure the attack surface is reduced as much as possible

Correct answer: a | LO: AS-8.2.1 (K4) | Syllabus §8.2.1, §5.4.2

Why: (a) — official rationale: **both of these techniques are used to test firewalls.** (b) and (c) **the goal is to prevent the attack rather than let it get through the firewall**. (d) **software component hardening will help the individual software components, but not the firewall and its implementation**. Syllabus §5.4.2: **using malformed network packets and network fuzzing to exploit an unexpected behavior (e.g. a Denial Of Service)**.

Q3. OFFICIAL · Sample Exam #43 (1 pt)

If you have acquired a tool that is used under the GNU General Public License, which of the following is an important consideration for tool maintenance?

a) Reliability of the vendor and ability to provide support

b) Frequency and availability of updates from the vendor

c) Technical capabilities of your team to support and customize the tool for your environment ✔

d) License cost and support contract cost

Correct answer: c | LO: AS-8.2.2 (K2) | Syllabus §8.2.2

Why: (c) — official rationale: **the GNU license is free and it is an open source community so there is no vendor.** (a) and (b) **because there is no vendor**. (d) **the tool is free although you may have development costs in customizing the tool**. Syllabus §8.2.2: **the organization may need to have technical capacity available for support and specific configuration**.

Part B — 12 additional syllabus-derived questions

*The K-level in each question's label is the level of the **learning objective** it practises, not of the question itself. Some Part B items are quick recall checks on a K3 or K4 objective. Use them to learn the facts, and use the scenario items (and Part A) to practise the reasoning the real K3/K4 questions need.*

Q4. Derived from syllabus

What is the main advantage of static analysis tools in security testing, according to the syllabus?

a) They detect memory leaks at run time

b) They can be used very early in the development lifecycle and are available for most languages, usually reporting on security aspects ✔

c) They probe the whole system including the network

d) They require no configuration

Correct answer: b | LO: AS-8.1.1 (K2) | Syllabus §8.1

Why: (b) — §8.1.

Q5. Derived from syllabus

A dynamic security testing tool performs a scan of a database's configuration while the application is not running. How does the syllabus describe this?

a) It is a misuse of the tool

b) It is purely dynamic testing

c) It is impossible

d) Such dynamic tools are then used as a kind of static testing tool — though if the entire system is the test object, they truly are dynamic tools ✔

Correct answer: d | LO: AS-8.1.1 (K2) | Syllabus §8.1

Why: (d) — §8.1: **a dynamic security test tool may perform a static scan of a database**.

Q6. Derived from syllabus

Why is selecting and using security testing tools a challenge?

a) Most tools are intelligent systems deploying non-standardized tests with different test data, implementations and interpretations, while the same defense or vulnerability can be implemented in different ways ✔

b) There are almost no security tools available

c) All tools give identical results, so differentiation is hard

d) Security tools cannot be automated

Correct answer: a | LO: AS-8.1.1 (K2) | Syllabus §8.1

Why: (a) — §8.1.

Q7. Derived from syllabus

What does the John the Ripper example illustrate?

a) That commercial tools are always better

b) That password cracking should never be done

c) That some hacker tools became the basis for legitimate security testing tools — it now detects weak Unix passwords ✔

d) That open source tools cannot be used by regulated organizations

Correct answer: c | LO: AS-8.1.1 (K2) | Syllabus §8.1

Why: (c) — §8.1.

Q8. Derived from syllabus

Which of the following is listed by the syllabus as a test basis for security testing?

a) The marketing plan

b) User profiles ✔

c) The office floor plan

d) Developer timesheets

Correct answer: b | LO: AS-8.2.1 (K4) | Syllabus §8.2.1

Why: (b). The list: security policy · test policy · threat and risk analysis results · requirements and specifications · architecture and design · security (test) strategy · the system under test · known threats, exploits and vulnerabilities · **user profiles**.

Q9. Derived from syllabus

A bank plans a new public REST API for partners, built in Java, deployed on hardened Linux servers behind a WAF. Security testing will continue after go-live as part of operations. The security policy requires TLS 1.2+ and the bank's web coding guidelines. The test team has scanner experience but no API tooling.

Which documented set of tool needs best follows the syllabus analysis?

a) Static analysis for Java against the web coding guidelines; API interface testing including input fuzzing; protocol/standard verification (TLS configuration); configuration verification of the hardened servers and WAF; and tooling suitable for ongoing operations use as well as development — noting the team's gap in API tooling ✔

b) A load testing tool to measure API throughput

c) A single GUI scanner for the partner portal only

d) No tools — manual review is sufficient

Correct answer: a | LO: AS-8.2.1 (K4) | Syllabus §8.2.1

Why: (a) derives needs from the syllabus overview: **interfaces to be tested**, **protocols and standards to be verified**, **web coding guidelines**, **system component configurations to be verified (hardened)**, and **whether security testing will be a development or a maintenance/operations activity**.

Q10. Derived from syllabus

What overview should requirements and design documents provide to help define tool requirements?

a) Budget and schedule

b) Names of the testers

c) Interfaces to be tested (including the GUI), protocols and standards to be verified, web coding guidelines, and system component configurations to be verified (hardened) ✔

d) List of open defects

Correct answer: c | LO: AS-8.2.1 (K4) | Syllabus §8.2.1

Why: (c) — §8.2.1.

Q11. Derived from syllabus

Why might an organization be unable to use open source security tools?

a) Open source tools cannot be modified

b) Open source tools never have a GUI

c) Open source tools are always less effective

d) Regulatory compliance issues can force organizations to use only commercial or otherwise certified tools ✔

Correct answer: d | LO: AS-8.2.2 (K2) | Syllabus §8.2.2

Why: (d) — §8.2.2.

Q12. Derived from syllabus

Which item belongs in an ROI calculation for an open source security tool?

a) The future direction of the tool — some tools may go away or go commercial ✔

b) The vendor's helpdesk hours

c) Floating licence pricing

d) The vendor's support contract

Correct answer: a | LO: AS-8.2.2 (K2) | Syllabus §8.2.2

Why: (a). The six aspects: limited scope · time to learn · time in forums · time to update/upgrade · **future direction** · community responsiveness. (b)–(d) are **vendor** attributes (§8.2.3).

Q13. Derived from syllabus

What mainly determines the number of security testing tool licences an organization needs?

a) The number of developers

b) The functional areas delivered by the tool, the frequency and time of use, and the number of security testers ✔

c) The number of servers in production

d) The number of open vulnerabilities

Correct answer: b | LO: AS-8.2.2 (K2) | Syllabus §8.2.2

Why: (b) — §8.2.2; **most businesses need one or a few**.

Q14. Derived from syllabus

Why is a vendor's update frequency especially important for security testing tools?

a) Because licences expire with each update

b) Because updates reduce licence costs

c) Because tools must stay current with constantly changing security threats ✔

d) Because updates replace the need for support contracts

Correct answer: c | LO: AS-8.2.3 (K2) | Syllabus §8.2.3

Why: (c) — AS-8.2.3: evaluate the vendor's capability **to update tools on a frequent basis to stay current with security threats**.

Q15. Derived from syllabus

Which of the following is a licence type listed among vendor attributes?

a) Floating ✔

b) Copyleft

c) Freeware

d) Shareware

Correct answer: a | LO: AS-8.2.3 (K2) | Syllabus §8.2.3

Why: (a) — **fixed / desk / floating / token**.

Most Important Questions From This Chapter

*Ranking rationale: guaranteed **AS-8.1.1 (K2)** and **AS-8.2.1 (K4)**; one K2 from 8.2.2 / 8.2.3.*

Rank	Topic / question focus	LO	Why it matters
1	Needs → tool capability — Q2, Q8–Q10	AS-8.2.1	**Guaranteed 3 points**; uses Chapter 5 tests.
2	Static vs dynamic security tools — Q1, Q4–Q7	AS-8.1.1	**Guaranteed 1 point.**

Rank	Topic / question focus	LO	Why it matters
3	Open source — Q3, Q11–Q13	AS-8.2.2	Pooled 1 point; no vendor.
4	Vendor evaluation — Q14, Q15	AS-8.2.3	Pooled 1 point; update frequency.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

LO	K	Pts	Sample #	Format used officially
AS-8.1.1	K2	1	#41	Dynamic security tools vs general → **probe the system**
AS-8.2.1	**K4**	**3**	#42	Firewall done + port scan; DoS worry → **malformed packets / fuzzing**
AS-8.2.2	K2	1	#43	GNU GPL tool maintenance → **your team's capability**
Chapter 8 total	—	**5**	**#41–#43**	**5.5 points per teaching hour — the best in the syllabus**

The arithmetic

5 points for 55 minutes. Learn this chapter early: it is the cheapest per hour, and its K4 reuses Chapter 5 firewall tests.

5.2 Common distractors observed

- **Vendor attributes for an open source tool** — there is no vendor.
- **Licence cost** for a GPL tool — it is free (customization costs aside).
- **Memory leaks** as a security-tool strength — that is general dynamic analysis.
- **Failover, performance or encryption speed** when the need is to **detect and reject** malformed traffic.
- **Component hardening** when the object is the **firewall**.

5.3 Keyword recognition (map words → answers)

If the stem says…	Think…
"GNU", "GPL", "open source"	**No vendor — team capacity**
"regulated", "certified tools"	Open source **may be forbidden**
"dynamic security tool"	**Probes the system**
"early in the lifecycle"	**Static analysis**
"John the Ripper"	Hacker tool → **legitimate weak-password detection**
"Nessus", "Metasploit"	Early **open source frameworks** with GUIs
"update frequency"	**Vendor evaluation** — stay current with threats
"fixed / desk / floating / token"	**Licence types**
"go commercial", "go away"	Open source **future direction** risk

5.4 Typical traps

- A **dynamic** security tool can run a **static** scan — static vs dynamic depends on whether the object is **operational**.
- **Commercial security tools are few** compared with open source.
- Testing may be a **development** or an **operations** activity — tool needs differ.

5.5 How to eliminate wrong answers

1. Ask **"is there a vendor?"** — strike vendor options for open source.
2. In need questions, strike options for **what is already done** and options aimed at a **different goal**.

5.6 Scenario-solving shortcuts

Scenario shape	Shortcut
Open source tool maintenance	**Your team supports and customizes**
Commercial tool choice	**Update frequency**, licences, support hours, community, manuals, contracts
Documented need list	Interfaces · protocols/standards · coding guidelines · configurations · dev vs ops

5.7 Common candidate mistakes

- Skipping this chapter as "just tools" — it is the best value per hour.
- Forgetting the firewall K4 lives here.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Expands to
"No vendor, your team"	Open source maintenance
S-L-F-U-F-R ("Some Learn Fast, Upgrade For Results")	Open source ROI: **S**cope limited, **L**earning time, **F**orum time, **U**pdate/upgrade time, **F**uture direction, **R**esponsiveness
L-S-H-F-U-M-C	Vendor: **L**icence types, **S**calability, **H**elpdesk, **F**orum, **U**pdate frequency, **M**anuals, **C**ontracts
I-P-W-C	Overview from requirements/design: **I**nterfaces, **P**rotocols & standards, **W**eb coding guidelines, **C**onfigurations

5.9 Chapter 8 mind map

Branch	Leaves
History	Hacking drove tools · John the Ripper · Nessus, Metasploit · usability vs complexity
Landscape	Massive, non-standardized · automate defense evaluation · detect known vulnerabilities · own list
Static/dynamic	Static early · dynamic probes the system · static scans by dynamic tools
Needs	Nine test-basis items · interfaces, protocols, guidelines, configurations · dev vs ops
Open source	Free, modifiable · regulation · team capacity · manuals · ROI aspects · licence count
Vendors	Licence types, scalability, support, community, update frequency, manuals, contracts

Section 6 — Revision Notes

6.1 One-page chapter summary

Security tools grew out of hacker tools (John the Ripper; Nessus and Metasploit frameworks). There are many, mostly open source, running non-standardized tests — so choosing is hard. Tools automate defense evaluation and detect known vulnerability types. Static tools work early; dynamic security tools probe the system and may even scan statically. Tool needs come from nine test-basis items and from the interfaces, protocols, coding guidelines and configurations they reveal, plus whether testing is a development or operations activity. Open source is free but needs your own technical capacity and may be ruled out by regulation; commercial vendors must be judged on licences, support, community, manuals, contracts — and above all **update frequency**.

6.2 Key definitions to memorize

Term	Definition
Static analysis (security)	Analysis while the object is not operational; early; most languages
Dynamic security tool	Probes the system rather than just the application
ROI aspects (open source)	Scope, learning, forums, upgrades, future direction, community

6.3 High-priority concepts (study first)

3. Needs → capability; firewall malformed/fuzz (3 pts).
4. Dynamic security tools probe the system (1 pt).
5. Open source = no vendor; vendor update frequency (1 pt).

6.4 Last-minute revision sheet

If you read nothing else

Dynamic security tools **probe the system**; static tools work **early**.
Firewall + DoS worry → **malformed packets / fuzzed data**.
GPL → **no vendor → your team**.
Vendor → **update frequency** (plus licences, support, community, manuals, contracts).
Needs: interfaces, protocols/standards, coding guidelines, configurations; dev vs ops.

6.5 Memorization checklist

✓	I can...
☐	explain static vs dynamic security tools including the static-scan twist
☐	list the nine test-basis items and four overview items
☐	list open source issues and the six ROI aspects
☐	list the seven vendor attributes and four licence types

6.6 Exam-day quick review

- **Probes the system** decides the static/dynamic question.
- **Need → capability** decides the K4; **is there a vendor?** decides the open-source/vendor question.